

SCANCHECK CLI

Document Perspective Correction and Image Quality Analysis

Course domain	Computer Vision
Project type	Command-line mini project
Student name	Devansh Kaushik
Registration number	24BAI10645
Submission date	18 September 2026

An offline and reproducible implementation using Python, OpenCV, and NumPy.
No training dataset, model download, GPU, GUI, or network connection is
required at runtime.

1. Abstract

Photographs of documents often contain perspective distortion, uneven lighting, and blur. ScanCheck CLI detects a paper document, transforms it into a top-down view, enhances readability, and calculates interpretable quality measurements. The system uses classical computer-vision techniques and requires neither training data nor specialized hardware. A deterministic sample generator and automated tests make the implementation straightforward to reproduce and evaluate.

2. Problem statement

An ordinary phone photograph is not always suitable for archiving or reading because a page may appear trapezoidal, dim, or unfocused. The project must locate a visible document, remove perspective distortion, enhance the corrected page, and report objective image-quality indicators from a terminal-only workflow.

3. Objectives

- Detect a document boundary using edges and contour geometry.
- Compute a projective transformation to obtain a rectangular top-down view.
- Enhance the result in black-and-white or color mode.
- Measure focus, exposure, contrast, and structural content.
- Provide reproducible inputs, outputs, tests, and complete setup instructions.

4. Scope

The project focuses on single-page photographs where all four page corners are visible. It performs document localization, geometric correction, enhancement, and quality assessment. Optical character recognition and multi-page PDF assembly are deliberately outside the current scope.

5. Tools and technologies

Component	Purpose
Python 3.10+	Application and command-line interface
OpenCV	Edges, contours, homography, enhancement, and metrics
NumPy	Point geometry and image arrays
pytest	Automated verification
Git / GitHub	Version control and public submission

6. System design

Input photograph -> grayscale and Gaussian blur -> Canny edges and morphological closing
-> largest convex quadrilateral -> corner ordering and perspective transform -> adaptive
threshold or CLAHE -> output scan and quality metrics

The command-line layer validates arguments and reports errors. The core module contains reusable image-processing functions. The demo module creates a known synthetic photograph so the complete pipeline can be run without external assets.

7. Project structure

```
scancheck/ - application source
tests/ - automated tests
docs/ - report files
requirements.txt - pinned runtime packages
requirements-dev.txt - test packages
pyproject.toml - package metadata and CLI entry point
README.md - evaluator instructions
```

8. Methodology

8.1 Pre-processing

The input is resized only for efficient detection, converted to grayscale, and smoothed with a 5 x 5 Gaussian kernel. Canny edge detection extracts strong intensity transitions. Morphological closing connects small breaks in the page boundary.

8.2 Document detection

Contours are sorted by area. Candidates covering less than 15 percent of the image are ignored. Each candidate is approximated using the Douglas-Peucker algorithm. The first large, convex four-point polygon is selected as the document.

8.3 Perspective correction

The four corners are ordered as top-left, top-right, bottom-right, and bottom-left. Target dimensions are estimated from opposite edge lengths. OpenCV computes a 3 x 3 homography and warps the source into a rectangular top-down view.

8.4 Enhancement

Black-and-white mode uses Gaussian adaptive thresholding, allowing local regions to receive appropriate thresholds under uneven illumination. Color mode converts to LAB space and applies contrast-limited adaptive histogram equalization to only the lightness channel.

8.5 Quality analysis

Metric	Definition	Interpretation
Blur score	Variance of Laplacian	Low value means weak fine detail
Brightness	Mean grayscale intensity	0 is black; 255 is white
Contrast	Grayscale standard deviation	Higher value means wider tonal spread
Edge density	Canny edge pixels / total pixels	Approximate structural content
Exposure	Brightness thresholds 70 and 190	Dark, balanced, or bright

9. Installation and execution

The evaluator opens a terminal at the repository root and runs:

```
python -m venv .venv
# Windows: .venv\Scripts\Activate.ps1
# macOS/Linux: source .venv/bin/activate
python -m pip install -r requirements.txt
python -m scancheck.cli demo
```

To scan a photograph:

```
python -m scancheck.cli scan path/to/photo.jpg --output scanned.png
python -m scancheck.cli scan path/to/photo.jpg --output scanned-color.png --color
```

To analyze quality:

```
python -m scancheck.cli analyze path/to/photo.jpg
python -m scancheck.cli analyze path/to/photo.jpg --json
```

10. Testing and results

Four automated tests verify point ordering, perspective-output dimensions, detection of a uniformly blurred image, and page detection in the generated demo photograph. The demo command exercises image generation, boundary detection, warping, enhancement, quality analysis, image writing, and JSON writing in one reproducible run.

```
python -m pip install -r requirements-dev.txt
python -m pytest
```

Expected demo outputs

- sample_photo.png - a deliberately perspective-distorted paper photograph.
- sample_scan.png - the detected page after correction and enhancement.
- quality.json - numerical image-quality measurements.

11. Error handling and reproducibility

Unreadable files and failed document detection produce a concise message and exit code 2. Successful commands exit with code 0. The sample is generated from fixed geometry with no randomness, dependencies are pinned, and the application makes no network calls at runtime.

12. Advantages

- Fully offline, lightweight, and independent of training data.
- Modular source code with reusable computer-vision functions.
- Human-readable and JSON quality output.
- Both high-contrast grayscale and color-preserving scan modes.
- Terminal operation suitable for automated evaluation.

13. Limitations

- Page boundaries must be visible against a sufficiently contrasting background.
- Strong shadows, folds, occlusion, or a larger rectangle can confuse contour selection.
- A fixed blur threshold cannot represent every camera and resolution.
- Text is enhanced but not recognized; OCR is not included.

14. Future enhancements

- Rank multiple candidates with geometric confidence scores.
- Add illumination normalization and shadow removal.
- Add optional OCR and searchable PDF export.
- Calibrate focus thresholds by resolution.
- Support folder batches and CSV summaries.

15. Conclusion

ScanCheck demonstrates a complete classical computer-vision pipeline in a compact and reproducible command-line project. It combines geometric vision, enhancement, and quality assessment while remaining simple to install, inspect, test, and evaluate.

16. References

1. OpenCV, *Image Processing in OpenCV*,
https://docs.opencv.org/4.x/d2/d96/tutorial_py_table_of_contents_imgproc.html
2. OpenCV, *Geometric Image Transformations*,
https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html
3. J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1986.
4. D. H. Douglas and T. K. Peucker, "Algorithms for the Reduction of the Number of Points Required to Represent a Digitized Line," *Cartographica*, 1973.

Submission checklist

- Replace the student name, registration number, and submission date on the cover page.
- Create a GitHub repository and upload the complete project root.
- Set the repository visibility to Public.
- Run the demo and tests once after cloning the public repository.
- Submit only a root URL in the form <https://github.com/username/repository-name>.
- Do not submit a `/tree/main` or `/blob` URL.